Voice Assistance for Multimedia Control

Hardware-Based Voice Control System using Raspberry Pi / ESP32

28 September 2026

Aaditya Vashisht: PES1UG24CS004
Aarohi Jaiswal: PES1UG24CS009
Abhay S: PES1UG24CS013

3. Software Architecture & Design Specification

1. Introduction

1.1 Purpose

This document describes the software architecture and detailed design of the Voice Assistance for Multimedia Control system. It explains how the components identified for the system work together, the architectural pattern used, and the detailed interaction and interface design needed to implement the requirements defined in the SRS.

1.2 Scope

This specification covers the internal software structure of the system running on the Raspberry Pi / ESP32 hardware: how audio input flows through recognition, security validation, and command execution, out to the media player and user feedback. It does not cover the internal design of the third-party media player application, only the interface used to control it.

1.3 References

- Software Requirements Specification (SRS) – Voice Assistance for Multimedia Control, v1.0.
- Software Test Plan – Voice Assistance for Multimedia Control, v1.0.
- IEEE Std 1016-2009, IEEE Standard for Information Technology – Software Design Descriptions.

2. Architecture

2.1 Architectural Overview

The system is organized into four layers that mirror the flow of a voice command through the system: a Hardware Layer (microphone input and speaker output), a Recognition Layer (audio capture, wake-word detection, speech-to-text), a Control Layer (command mapping, security validation, command execution, and error handling), and an Output Layer (the media player interface and user feedback/LED indicator).

2.2 Component Diagram and Description

Figure 1 shows the component structure of the system, grouped by layer, with arrows indicating the direction of data/control flow between components.

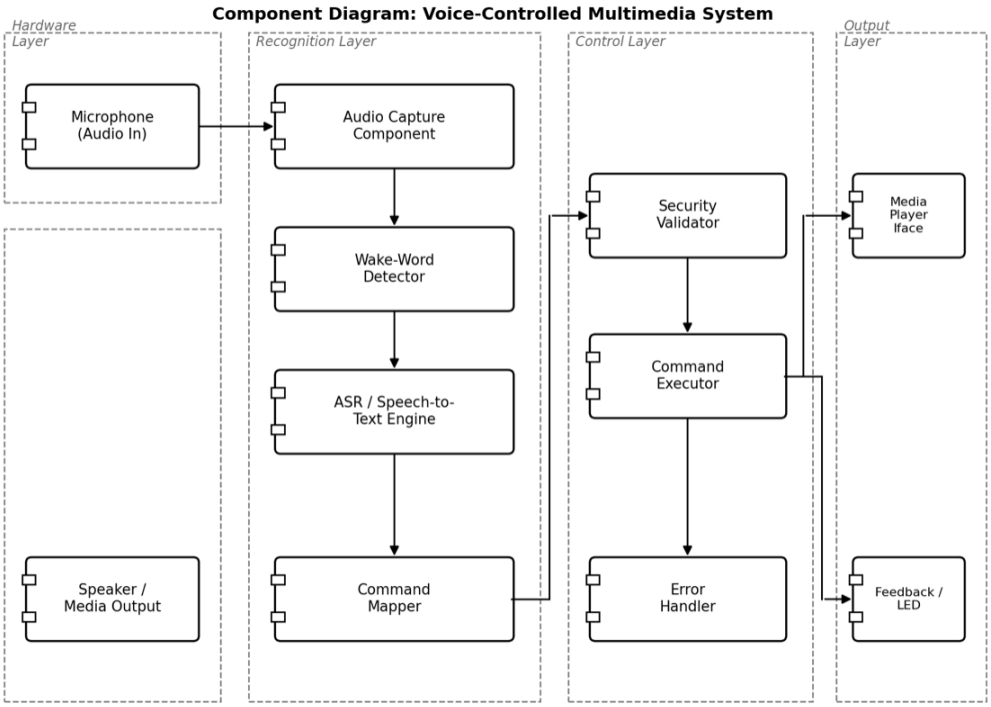


Figure 1: Component Diagram — Voice-Controlled Multimedia System

Component	Responsibility
Audio Capture Component	Continuously samples audio from the microphone into a rolling buffer for the Wake-Word Detector.
Wake-Word Detector	Monitors the buffered audio for the defined wake word; on detection, opens a listening window and forwards subsequent audio to the ASR Engine (implements SR-1).
ASR / Speech-to-Text Engine	Converts the captured audio into text using an offline/lightweight speech recognition model.
Command Mapper	Matches recognized text against the defined command vocabulary (Play, Pause, Resume, Next, Previous, Volume Up, Volume Down) and produces a structured command object.
Security Validator	Confirms the command originated from the authorized wake-word-gated session before allowing execution (implements SO-1, SR-1).
Command Executor	Invokes the appropriate action on the Media Player Interface and requests user feedback on success.

Error Handler	Catches unrecognized commands or execution failures and logs them without performing any media action.
Media Player Interface	Thin adapter exposing play(), pause(), resume(), next(), previous(), volume_up(), volume_down() to the Command Executor, mapped to the actual media player's control API.
Feedback / LED Module	Provides an audible or visual acknowledgement to the user once a command has executed (or a short tone if a command was rejected).

2.3 Architectural Pattern

The system follows a Layered (Pipe-and-Filter) architecture. Audio data flows in one direction through a pipeline of filters — capture, wake-word detection, recognition, mapping, validation, execution — with each stage consuming the previous stage's output and producing input for the next. This pattern was chosen because the problem is naturally a linear transformation of a signal (raw audio) into a discrete action (a media command), and because it keeps each stage independently testable, matching the modular test cases defined in the Test Plan.

2.4 Traceability to Requirements

Each architectural component is traced back to the requirement(s) it satisfies, ensuring the design fully covers the SRS:

Component(s)	Requirement ID(s)
Audio Capture Component / Wake-Word Detector	FR-1
Command Mapper + Command Executor	FR-2 to FR-8
Error Handler	FR-9
Feedback / LED Module	FR-10
Wake-Word Detector, ASR Engine (offline model)	NFR-1, NFR-2
Overall pipeline (stateless, always-listening design)	NFR-3
Security Validator	SO-1, SR-1
Local ASR Engine (no mandatory cloud call)	SO-2, SR-2

2.5 Security Architecture

Security is addressed at two points in the pipeline rather than as an afterthought:

- Wake-word gating: the Wake-Word Detector only opens the listening window for a short, fixed duration after detecting the exact wake word, which limits the system's attack surface against ambient or unintended audio (SO-1, SR-1).
- Local-only processing: the ASR Engine and all downstream components run on-device; no raw audio is sent off the Pi/ESP by default, which protects against interception of audio data over a network (SO-2, SR-2).
- The Security Validator acts as a final gate immediately before the Command Executor, so even if a downstream component were compromised or misbehaved, no command reaches the media player without passing validation.

2.6 Deployment View

All components run as a single Python process (or process group, for the ESP32 firmware variant) on one embedded board. There is no client-server split and no mandatory network dependency for core operation; an optional Wi-Fi connection may be used only for future OTA (over-the-air) updates, which is outside the current scope.

3. Design

This section presents the detailed, interaction-level design of the system: two UML sequence diagrams covering the primary (happy-path) and error-handling flows, the internal API design, and error-handling strategy.

3.1 Sequence Diagram 1 – Play Command Execution Flow

Figure 2 illustrates the end-to-end flow when a user issues a valid "Play" command, including the security validation step before execution and the acknowledgement returned to the user. The same flow applies, with the command name substituted, for Pause, Resume, Next, Previous, Volume Up, and Volume Down (FR-3 to FR-8).

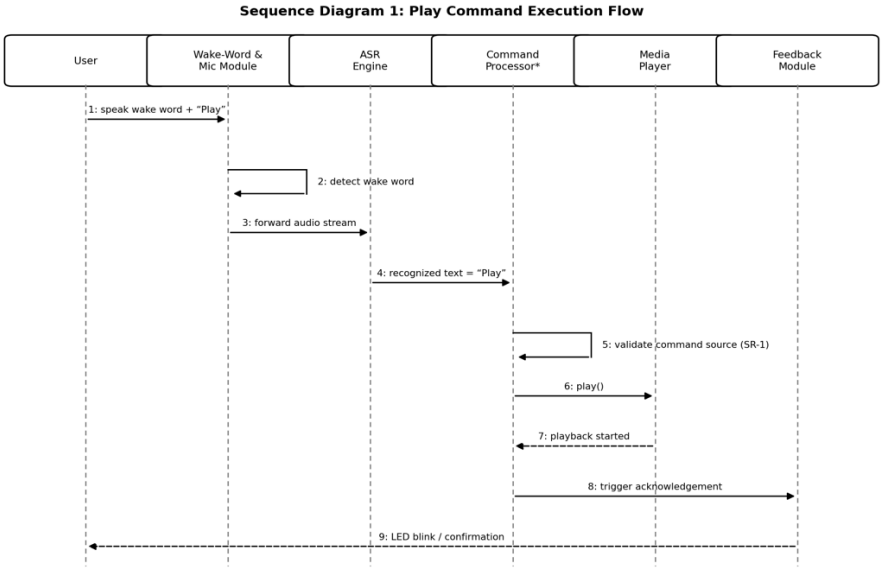


Figure 2: Sequence Diagram 1 — Play Command Execution Flow

3.2 Sequence Diagram 2 – Unrecognized Command / Error Handling Flow

Figure 3 illustrates the flow when audio is captured after the wake word but does not match any command in the vocabulary. The Error Handler intercepts this case so that no unintended action is ever executed on the media player (FR-9).

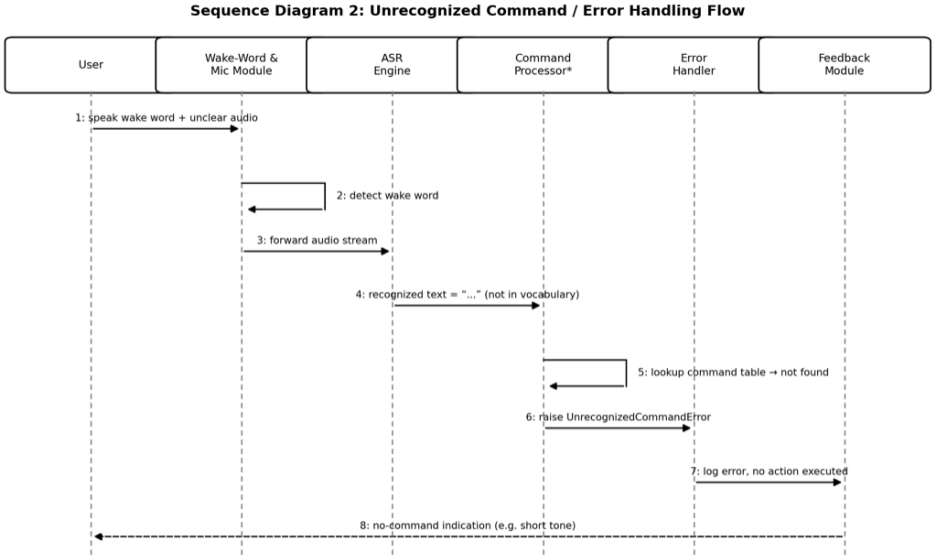


Figure 3: Sequence Diagram 2 — Unrecognized Command / Error Handling Flow

3.3 API Design

The table below defines the internal module-level API used between components. All calls are local function calls within the same process (no network API is required for core functionality).

Function	Owner Component	Parameters	Returns	Notes
detect_wake_word(audio_chunk)	Wake-Word Detector	audio_chunk: bytes	bool – True if wake word present	Called continuously on each incoming audio frame.
recognize_speech(audio_stream)	ASR Engine	audio_stream: bytes	text: str	Called once per listening window opened by the wake-word detector.
map_command(text)	Command Mapper	text: str	Command object {type, confidence}	Returns null/None if text does not match the defined vocabulary.
validate_source(command)	Security Validator	command: Command	bool – True if authorized	Checks the command originated within an active, wake-word-gated session.
execute(command)	Command Executor	command: Command	ExecutionResult {success, message}	Dispatches to the matching Media Player Interface method.
play() / pause() / resume() / next() / previous() / volume_up() / volume_down()	Media Player Interface	none	bool – action success	Thin wrappers over the underlying media player's own control API.
handle_error(error)	Error Handler	error: Exception	none (logs only)	Logs the error and triggers a neutral feedback tone; never raises further.
notify(status)	Feedback / LED Module	status: 'ok' | 'error'	none	Drives the LED / tone output

				visible or audible to the user.

3.4 Error Handling

All errors are funneled through the central Error Handler component so that failure behaviour is consistent and never results in an unintended media action. The table below summarizes the handled error conditions:

Error Condition	Detected By	System Response	Related Req.
No wake word detected	Wake-Word Detector never opens a listening window	System remains idle; no downstream component is invoked	SR-1
Unrecognized command (out of vocabulary)	Command Mapper returns no match for the recognized text	Error Handler logs the event; Feedback Module plays a short “not understood” tone; no media action occurs	FR-9
ASR recognition failure / timeout	ASR Engine raises a timeout or empty-result error	Error Handler logs the failure; system returns to idle listening state	NFR-1, NFR-3
Security validation failure	Security Validator returns False for a command	Command Executor is never called; Error Handler logs a security event	SR-1, SO-1
Media player unresponsive	Media Player Interface call fails or times out	Command Executor reports failure to Error Handler; Feedback Module signals an error tone instead of a success tone	FR-10
Microphone / hardware disconnected	Audio Capture Component receives no input stream	System logs a hardware fault and suspends listening until the input stream is restored	NFR-3

3.5 Data Design

Two lightweight data structures flow through the pipeline:

- Audio Buffer: a short rolling buffer of raw audio samples (a few seconds) used by the Wake-Word Detector and ASR Engine; it is discarded/overwritten continuously and never persisted to disk, consistent with the local-processing security requirement (SR-2).
- Command object: a simple structure {type: string, confidence: float, timestamp: datetime} produced by the Command Mapper and passed through the Security Validator to the Command Executor. No user-identifying data is stored in this object.